AI Usage Log - Sprint 2: Ismail Cherfaoui

Task 5.3: Define the Job Interface for the Job Listing UI

Purpose of AI use: Verifying TypeScript interface syntax and brainstorming data fields for the `Job` type used by the job search and listing feature (US-05).

Chat link: Claude Opus 5.5 (High)

AI suggested content: Claude confirmed the `export interface` syntax (including optional and read only properties) and suggested the following additional fields:

- `recruiterId`
- `postedAt`
- `deadline`
- `employmentType (full-time/part-time/internship/contract)`
- `workMode (on-site/remote/hybrid)`
- `salaryMin`
- `salaryMax`
- `skills`
- `status (open/closed)`

Validation: The suggested fields were compared against the Sprint 2 specifications. They were also checked against the existing conventions in `api.ts`.

Decision: Modified and partially accepted. `posted_at`, `employment_type`, `work_from` were retained and converted to snake case. The other suggested fields belong to the job posting schema (US-04, 4.1) and will be discussed with the team.

Reflection: The field suggestions were good, but they did not follow our naming conventions, and some of the suggested fields do not belong in this schema, so I only kept those relevant to my implementation.

Responsible person: Ismail Cherfaoui

Task 5.3: Generate Sample job Listings for the Job Listing UI

Purpose of AI use: Generating sample job listings to display in the job listing UI until the SQL table and the API endpoint exist.

Chat link: Claude Opus 5.5 (High)

AI suggested content: Claude generated 10 fictional job listings matching the `Job` interface.

Validation: The listings' data types were checked against the `Job` interface.

Decision: Accepted.

Reflection: The listings saved time writing sample data by hand and they can later be reused as data in the tables.

Responsible person: Ismail Cherfaoui

Task 5.3: Style the Job Listing Cards

Purpose of AI use: Generating CSS for the job listing cards, and confirming how to import a CSS file in React.

Chat link: Claude Opus 5.5 (High)

AI suggested content: Claude wrote CSS for the two new classes used in the job posting page (`cc-jobs-list` and `cc-job-card`) removing list bullets, adding a responsive grid, card borders, and a hover effect. It also confirmed that CSS is directly imported using `import "<filepath>"`.

Validation: The CSS was tested in the app. It displays correctly in dark mode, but hardcoded values caused issues in light mode.

Decision: Modified and accepted. The overall layout (removing bullets, grid) was kept. The styles were placed in a new `styles/jobs.css` file. The colours were reworked to match the app's existing theme variables, and the look and layout were adjusted to match the rest of the dashboard.

Reflection: The layout suggestions were a good starting point, but the style needed to be tweaked to support both themes and to stay consistent with the rest of the website.

Responsible person: Ismail Cherfaoui

Task 5.3: Display Labels for Job Type and Work Mode

Purpose of AI use: Finding how to display stored values (such as `full_time`) as a readable label in React.

Chat link: Claude Opus 5.5 (High)

AI suggested content: Claude suggested lookup maps typed with `Record` on the interface's union types.

Validation: Tested in the app, the labels displayed correctly.

Decision: Accepted, later removed. After aligning with the backend schema, the stored values were already formatted properly for display, so this implementation was no longer needed.

Reflection: This approach worked well, but matching the backend values made it unnecessary.

Responsible person: Ismail Cherfaoui

Task 5.1: Implement the Job Search API Endpoint

Purpose of AI use: Generating the `GET /api/jobs` search route and debugging a 401 error while testing it.

Chat link: Claude Opus 5.5 (Medium)

AI suggested content: Claude wrote the route in `routers/job_search.py`: open jobs only, newest first, optional keyword search, partial filters for location and category, and exact filters for job type and

work mode. It also gave the two lines to register the router in `app.py` and helped me test the endpoint.

Validation: The route was tested with a logged-in token using `curl` commands. It returned `200 OK` with correct filtering, and `401 Unauthorized` without a token.

Decision: Accepted.

Reflection: The route followed the existing style. Search tests were added afterwards.

Responsible person: Ismail Cherfaoui

Tasks 5.3 and 5.4: Rework Job Dashboard Tab with Search Filters and Details View

Purpose of AI use: Reworking `JobDashboardTab.tsx` to fetch jobs from the API using listJobs, adding filter inputs, handling loading/error/empty states, and adding an expandable details view to the jobs card.

Chat link: Claude Opus 5.5 (Low)

AI suggested content: Claude rewrote `JobDashboardTab.tsx` to fetch jobs with `listJobs`, added a 350ms delay on filter inputs, and made the job cards expand in place to show full details using the existing jobs.css classes.

Validation: Tested the filters, clear button, and expandable cards in the app.

Decision: Accepted.

Reflection: Expanding the cards in place worked well with my existing CSS.

Responsible person: Ismail Cherfaoui

Task 5.1: Write Pytest Tests for the Job Search Endpoint

Purpose of AI use: Generating a `pytest` test file (`test_job_search.py`) for the `GET /api/jobs` search endpoint.

Chat link: Claude Opus 5.5 (Low)

AI suggested content: Claude wrote `test_job_search.py` using the same structure as `test_jobs.py`. It used random tags in job titles so that tests do not interfere in the shared database, and covered login checks, newest first ordering, hiding closed jobs, search filters, and errors on invalid inputs.

Validation: The tests were run with `pytest` and all passed.

Decision: Accepted.

Reflection: The generated tests matched our existing test style, and the random tags keep the tests independent of each other in the shared database.

Responsible person: Ismail Cherfaoui

Task S2.1: Write Acceptance Tests for Sprint 1 and 2 User Stories

Purpose of AI use: Generating acceptance tests in Given/When/Then format for user stories US-01 to Us-07 to add as GitHub issues.

Chat link: Claude Opus 5.5 (Low)

AI suggested content: Claude generated two acceptance tests per user story, matching the format I gave it for US-05, and linking the relevant user story and task issue numbers for clarity.

Validation: The tests were checked against the user stories' requirements and the GitHub issue numbers.

Decision: Accepted.

Reflection: The acceptance tests were relevant and simple, as was requested in the format. The same chat can later be used to generate more acceptance tests for future Sprints.

Responsible person: Ismail Cherfaoui